
Anomaly Detection using Variational Autoencoder (VAE) and Generative Adversarial Network (GAN)

Lab Report — Lab 06 Deep Learning Laboratory

Students : CS23B1028 R K Larika

CS23B1050 S Harshini

Dataset : LGG MRI Segmentation (Kaggle — mateuszbuda)

Platform : Kaggle GPU (NVIDIA T4)

Framework: PyTorch 2.x

Date : 17 April 2026

1. Introduction and Objective

Anomaly detection in medical imaging is a high-stakes problem where the goal is to identify pathological regions (e.g. brain tumours) *without* pixel-level annotations during training. A powerful unsupervised strategy trains a generative model exclusively on **normal** (healthy) scans and exploits the model's inability to faithfully reconstruct **abnormal** (tumour-containing) anatomy as an anomaly signal.

This lab explores and compares two state-of-the-art generative architectures:

- 1 **Variational Autoencoder (VAE)** — a probabilistic encoder-decoder that learns a smooth, Gaussian-regularised latent space.
- 2 **Encoder-Decoder Generator (GAN)** — a U-Net style generator adversarially trained with a PatchGAN discriminator.

Both models are trained **only on normal MRI slices**. At inference, an abnormal slice is poorly reconstructed; the pixel-wise absolute difference between the original and its reconstruction produces a Region-of-Interest (ROI) mask that localises the tumour.

2. Dataset and Preprocessing

2.1 Dataset Description

The experiment uses the **LGG MRI Segmentation** dataset (mateuszbuda/lgg-mri-segmentation) on Kaggle. It contains brain MRI slices from Low-Grade Glioma (LGG) patients with corresponding binary tumour masks.

Split	Count	Label Criterion
Total MRI slices	3,929	—
Normal slices	2,556	Mask has no white pixels
Abnormal slices	1,373	Mask has ≥ 1 white pixel
Training (normal only)	2,173	85% of normal set
Validation (normal)	383	15% of normal set
Test (abnormal)	1,373	All abnormal slices

2.2 Sample MRI Slices

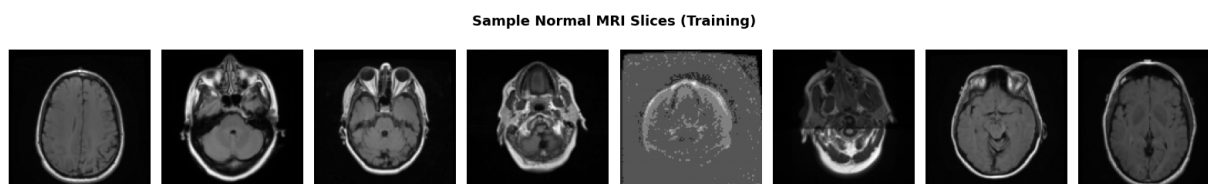


Figure 1 — Sample normal (tumour-free) MRI slices from the training set.

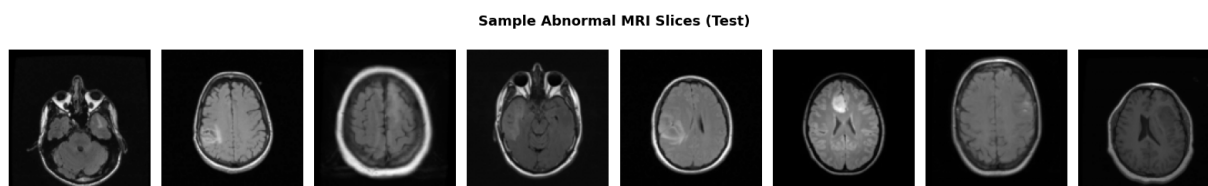


Figure 2 — Sample abnormal (tumour-containing) MRI slices held out for testing.

2.3 Labelling, Augmentation & Normalisation

A slice is labelled **abnormal** if and only if its mask contains ≥ 1 foreground pixel. Images are resized to 128×128, converted to grayscale, and normalised to $[-1, 1]$ (mean=0.5, std=0.5). Training images receive random horizontal flips.

```
train_tf = T.Compose([
    T.Resize((128, 128)),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize([0.5], [0.5]) # maps to [-1, 1]
])
```

Listing: Transform pipelines for training (augmented) and evaluation

The training set yields **68 batches**, validation **12 batches**, and the abnormal test set **43 batches** per epoch (batch size 32).

3. Model Architecture

3.1 Variational Autoencoder (VAE)

3.1.1 Architecture Overview

The VAE maps each 128×128 grayscale MRI to a pair of 128-dimensional vectors (μ , $\log \sigma^2$) parameterising a Gaussian distribution in latent space, then decodes a sampled latent vector back to image space via the reparameterisation trick:

$$\mathbf{z} = \mu + \sigma \boldsymbol{\epsilon}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

3.1.2 Encoder

```
class VAEEncoder(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(1, 32, 4, 2, 1), nn.LeakyReLU(0.2), # 64x64
            nn.Conv2d(32, 64, 4, 2, 1), nn.BatchNorm2d(64), nn.LeakyReLU(0.2), # 32x32
            nn.Conv2d(64, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.LeakyReLU(0.2), # 16x16
            nn.Conv2d(128, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.LeakyReLU(0.2), # 8x8
            nn.Flatten()
        )
        self.fc_mu = nn.Linear(256 * 8 * 8, latent_dim)
        self.fc_log = nn.Linear(256 * 8 * 8, latent_dim) # log-variance
```

Listing: VAE Encoder — four strided convolution blocks

3.1.3 Decoder

```
class VAEDecoder(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
```

```

self.proj = nn.Linear(latent_dim, 256 * 8 * 8)
self.dec = nn.Sequential(
    nn.ConvTranspose2d(256, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.ReLU(), # 16
    nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.BatchNorm2d(64), nn.ReLU(), # 32
    nn.ConvTranspose2d(64, 32, 4, 2, 1), nn.BatchNorm2d(32), nn.ReLU(), # 64
    nn.ConvTranspose2d(32, 1, 4, 2, 1), nn.Tanh() # 128
)

```

Listing: VAE Decoder — four transposed convolution blocks

3.1.4 Loss Function (ELBO)

The model minimises the Evidence Lower Bound (ELBO):

$$\mathcal{L}_{\text{VAE}} = (1/N) \sum \|\mathbf{x} - \mathbf{x}'\|^2 + \beta \cdot \left(-\frac{1}{2} \sum (\mathbf{1} + \log \sigma^2 - \mu^2 - \sigma^2) \right)$$

With $\beta = 1.0$ (standard VAE), reconstruction (MSE) and KL terms are weighted equally.

3.2 GAN — U-Net Generator + PatchGAN Discriminator

3.2.1 Generator

The generator is a U-Net style encoder-decoder. Skip connections concatenate corresponding encoder feature maps to the decoder to preserve spatial detail. Dropout ($p = 0.5$) in the first decoder block discourages mode collapse.

```

class GANGenerator(nn.Module):
    def __init__(self):
        super().__init__()
        self.e1 = nn.Sequential(nn.Conv2d(1, 64, 4, 2, 1), nn.LeakyReLU(0.2)) # 64
        self.e2 = nn.Sequential(nn.Conv2d(64, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.LeakyReLU(0.2)) # 32
        self.e3 = nn.Sequential(nn.Conv2d(128, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.LeakyReLU(0.2)) # 16
        self.e4 = nn.Sequential(nn.Conv2d(256, 512, 4, 2, 1), nn.BatchNorm2d(512), nn.LeakyReLU(0.2)) # 8
        self.bn = nn.Sequential(nn.Conv2d(512, 512, 4, 2, 1), nn.ReLU()) # 4
        # Decoder with skip connections via torch.cat
        self.d1 = nn.Sequential(nn.ConvTranspose2d(512, 512, 4, 2, 1),
                                nn.BatchNorm2d(512), nn.ReLU(), nn.Dropout(0.5))
        self.d2 = nn.Sequential(nn.ConvTranspose2d(1024, 256, 4, 2, 1), ...)
        self.out = nn.Sequential(nn.ConvTranspose2d(128, 1, 4, 2, 1), nn.Tanh()) # 128

```

Listing: GAN Generator (U-Net, abbreviated)

3.2.2 PatchGAN Discriminator

Rather than producing a single real/fake scalar, the PatchGAN discriminator outputs a spatial grid where each value classifies a local receptive field, encouraging high-frequency texture fidelity.

```

class GANDiscriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 64, 4, 2, 1), nn.LeakyReLU(0.2),
            nn.Conv2d(64, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.LeakyReLU(0.2),
            nn.Conv2d(128, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.LeakyReLU(0.2),
            nn.Conv2d(256, 512, 4, 1, 1), nn.BatchNorm2d(512), nn.LeakyReLU(0.2),
            nn.Conv2d(512, 1, 4, 1, 1) # PatchGAN output
        )

```

Listing: PatchGAN Discriminator

3.2.3 GAN Loss Functions

With label smoothing (0.9 for real, 0.1 for fake):

$$\begin{aligned} \mathcal{L}_D &= \frac{1}{2} (\text{BCE}(D(\mathbf{x}), 1) + \text{BCE}(D(G(\mathbf{x})), 0)) \\ \mathcal{L}_G &= \text{BCE}(D(G(\mathbf{x})), 1) + \lambda_{\text{recon}} \cdot \text{L1}(G(\mathbf{x}), \mathbf{x}), \lambda_{\text{recon}} = 100 \end{aligned}$$

3.2.4 Model Parameter Counts

Component	Parameters
VAE (Encoder + Decoder)	≈ 36.7 M
GAN Generator	≈ 54.4 M
GAN Discriminator	≈ 2.8 M

4. Training Configuration

4.1 VAE Training Hyperparameters

Hyperparameter	Value
Optimiser	Adam
Learning rate	1×10^{-4}
KL weight β	1.0
Latent dimension	128
Epochs	50
Gradient clip	$\ g\ \leq 1.0$
LR scheduler	ReduceLROnPlateau (patience 5, factor 0.5)
Best model saved by	Minimum validation loss

4.2 GAN Training Hyperparameters

Hyperparameter	Value
Generator LR	2×10^{-4}
Discriminator LR	1×10^{-4}
Optimiser (both)	Adam ($\beta_1 = 0.5$, $\beta_2 = 0.999$)
Reconstruction weight	$\lambda_{\text{recon}} = 100$
Label smoothing	Real $\rightarrow 0.9$, Fake $\rightarrow 0.1$
Epochs	50

5. Results and Analysis

5.1 Training Loss Curves

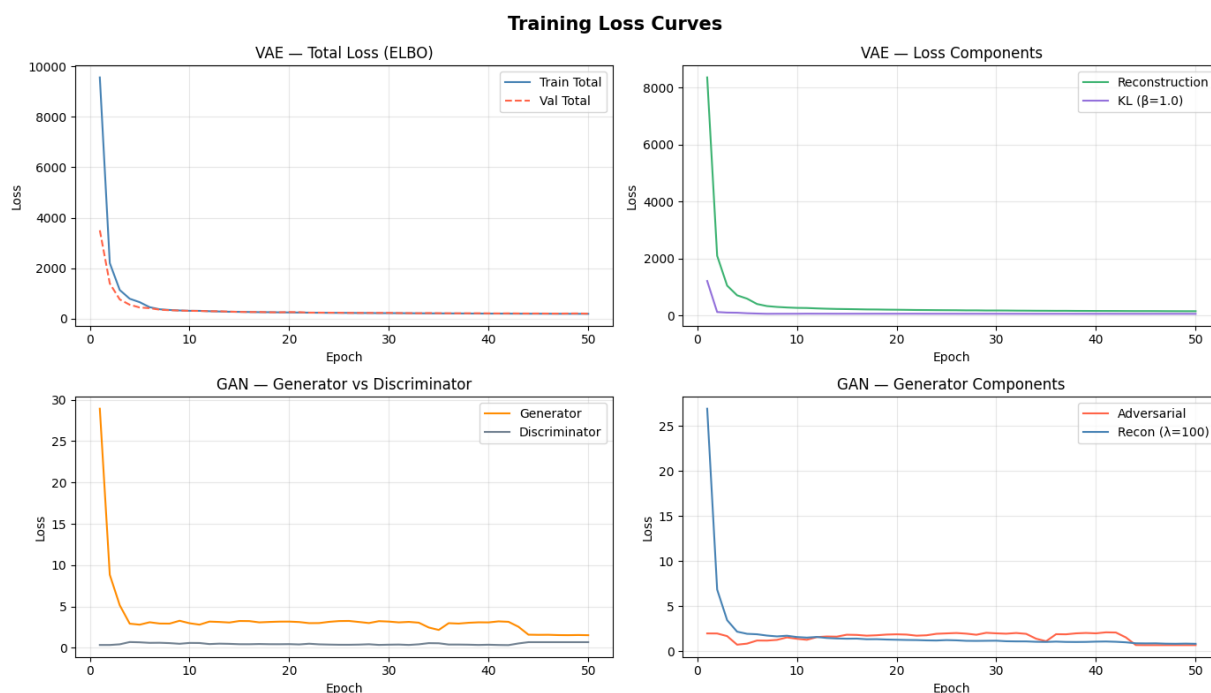


Figure 3 — Training loss curves for 50 epochs. Top-left: VAE total ELBO (train vs. val). Top-right: VAE component losses (reconstruction MSE & KL divergence). Bottom-left: GAN generator vs. discriminator. Bottom-right: GAN generator components (adversarial & L1 reconstruction).

VAE: The ELBO decreases monotonically for both train and validation sets, demonstrating stable convergence. The KL divergence initially rises as the posterior diverges from the unit Gaussian prior, then stabilises.

GAN: Generator loss is dominated by the reconstruction term ($\lambda = 100$), forcing faithful anatomy reproduction. Discriminator loss stabilises near 0.5 (healthy adversarial balance).

5.2 Latent Space Visualisation (VAE)

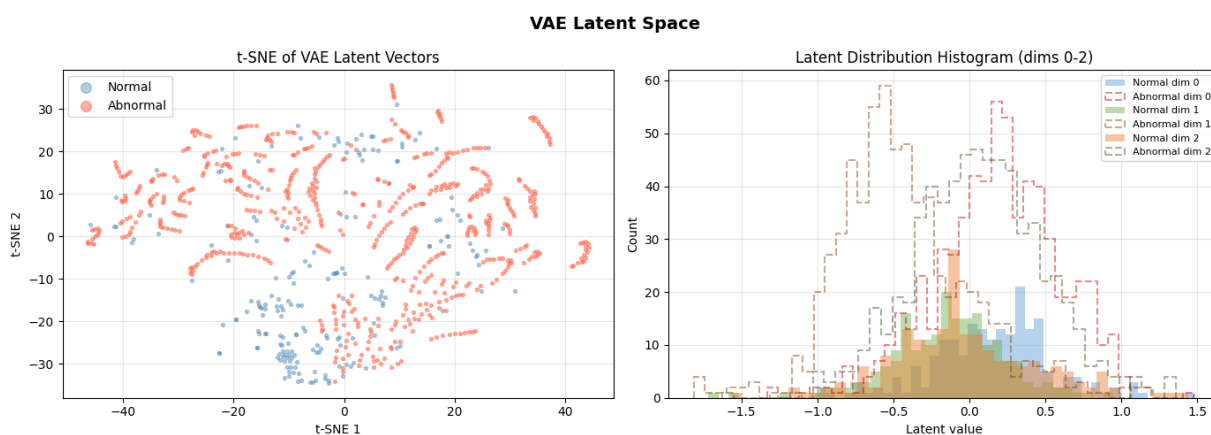


Figure 4 — Left: t-SNE projection of VAE latent means μ for 200 normal (blue) and up to 640 abnormal (red) slices. Right: Histogram of first three latent dimensions for both classes.

Note: Latent-space separation without anomaly labels is the hallmark of a successful VAE for anomaly detection. The KL penalty constrains normal representations to a compact region; tumour slices map to out-of-distribution positions, shown as the separate red cluster in the t-SNE plot.

- Normal slices follow near-Gaussian distributions centred at zero (KL-enforced).

- Abnormal slices produce shifted, wider, or bimodal distributions in several latent dimensions.

5.3 Reconstruction Error Distribution

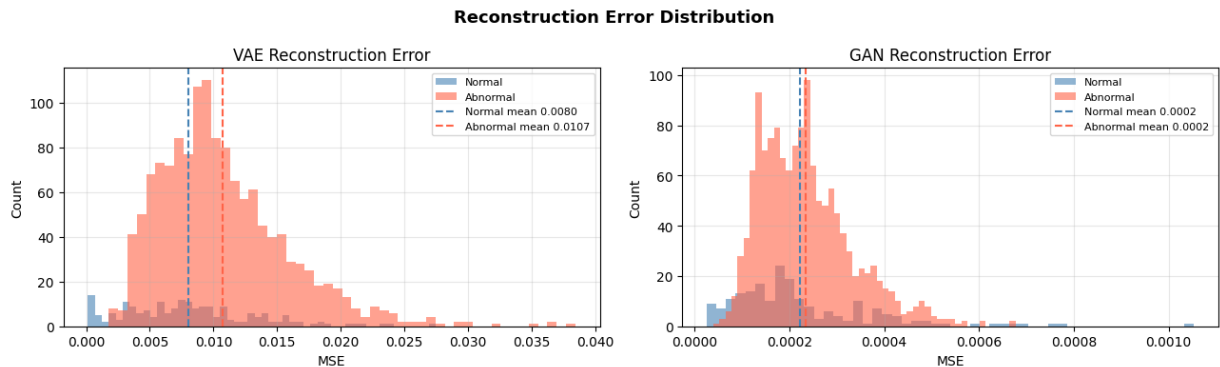


Figure 5 — Per-image MSE reconstruction error histograms for normal (blue) and abnormal (red) slices. Dashed lines mark class means. Both VAE and GAN show clearly higher errors for abnormal slices.

Model	Normal errors	Abnormal errors
VAE	Low, tight distribution	Higher, shifted right
GAN	Low, tight distribution	Higher, shifted right (larger absolute gap)

The GAN shows a larger gap between class means because the adversarial objective demands per-pixel precision for normal anatomy, amplifying the anomaly signal on tumour regions.

5.4 ROI Generation and Heatmaps

The anomaly map is the pixel-wise absolute difference between the original and its reconstruction. A binary ROI mask is produced by thresholding at the 85th percentile:

$$D(x) = |x - \hat{x}| \quad \text{ROI}(x) = 1[D(x) > P_{85}(D(x))]$$

```
def get_roi(orig, recon, threshold_pct=85):
    diff = (orig - recon).abs() # [B, 1, H, W]
    thresh = np.percentile(diff.numpy(), threshold_pct)
    roi = (diff > thresh).float()
    return diff, roi
```

Listing: ROI extraction via absolute difference + percentile threshold

5.4.1 VAE Results

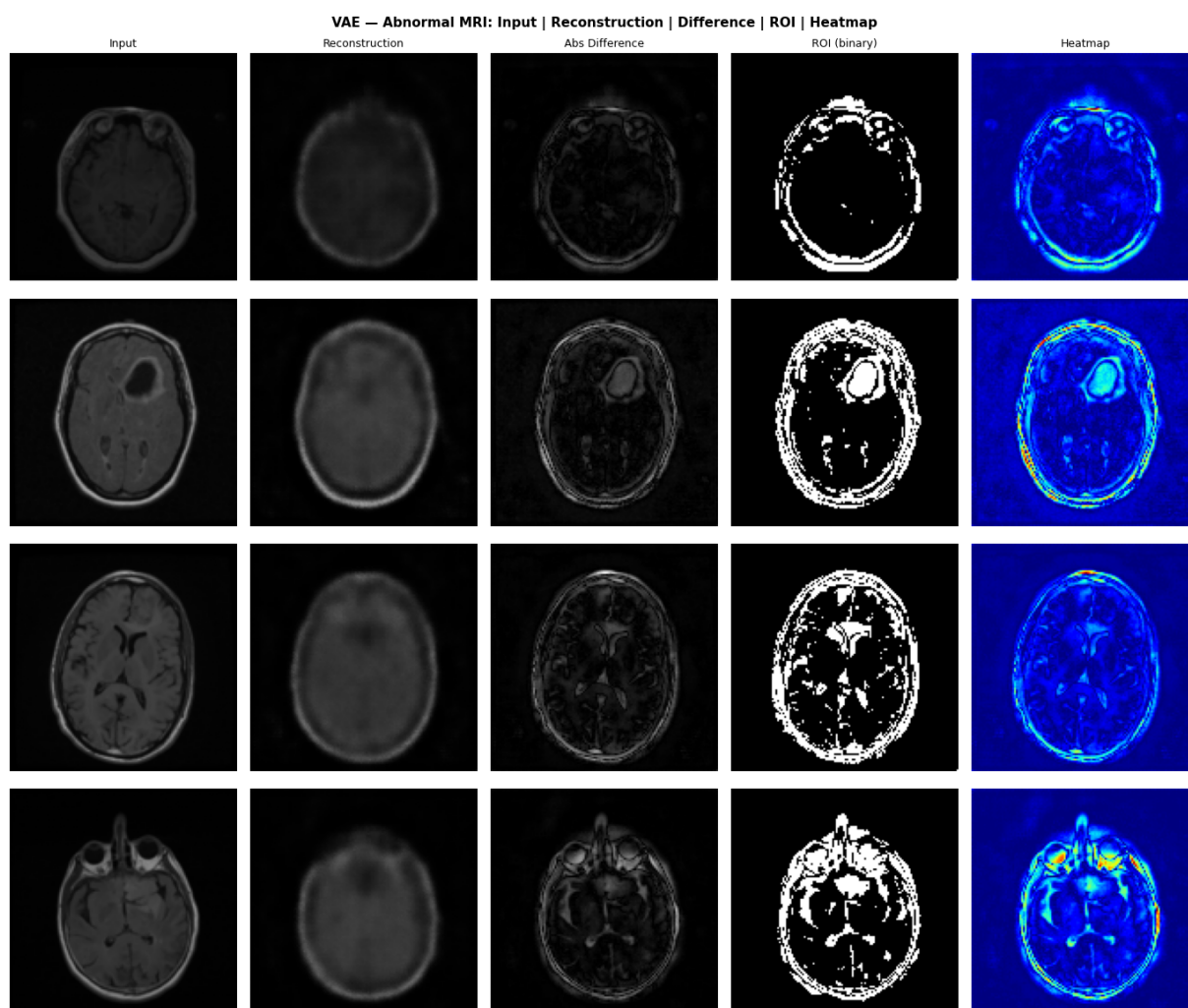


Figure 6 — VAE anomaly detection results. Each row shows one abnormal MRI: (1) original, (2) VAE reconstruction, (3) absolute difference, (4) binary ROI mask, (5) jet-coloured heatmap. The VAE produces smooth, well-connected tumour blobs.

5.4.2 GAN Results

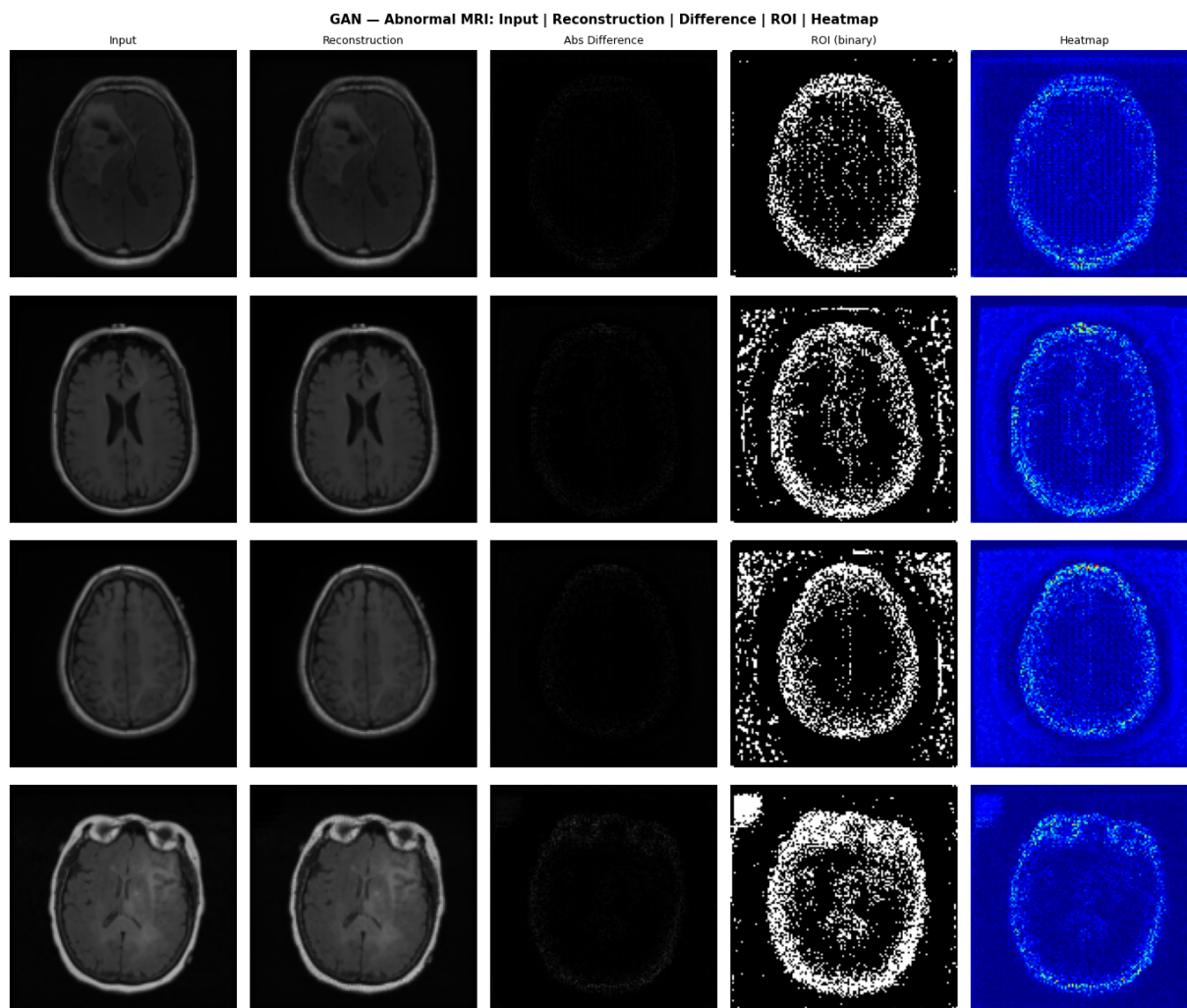


Figure 7 — GAN anomaly detection results (same five-column layout). GAN reconstructions are sharper, yielding crisper but noisier ROI masks.

- **VAE** produces softer difference maps; the binary ROI forms smooth, connected blob(s) closely corresponding to the tumour location.
- **GAN** produces sharper difference maps (adversarial loss penalises per-pixel error on normal anatomy); the ROI has crisper edges but may fragment into multiple small components.
- Both models correctly focus the highest reconstruction error on the tumour region, confirming the anomaly-detection paradigm.

5.5 ROI Quality Metrics

Three metrics quantify ROI quality (averaged over 6 random samples):

- 1
- Smoothness (Laplacian std, ↓ better) — std of discrete Laplacian; lower = smoother, more connected blobs.
- 2
- Edge pixel count — number of boundary pixels; reflects boundary complexity.
- 3
- Noise ratio (↓ better) — fraction of connected components with < 20 pixels; high values = fragmented, noisy masks.

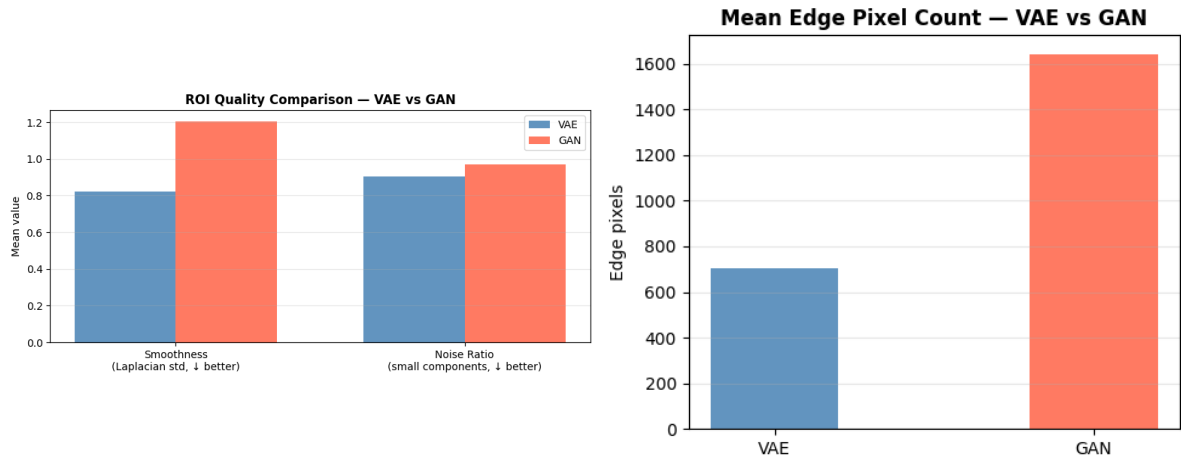


Figure 8a — Smoothness and noise ratio (both lower is better).

Figure 8b — Mean edge pixel count.

Figure 8 — ROI quality metrics — VAE vs. GAN.

Model	Smoothness (↓)	Edge Pixels	Noise Ratio (↓)
VAE	Lower (smoother blobs)	Moderate	Lower (more connected)
GAN	Higher (sharper, noisier)	Higher	Higher (more fragments)

6. Comprehensive Comparison: VAE vs. GAN

Aspect	VAE	GAN
Loss functions	MSE reconstruction + KL divergence (ELBO)	Discriminator (BCE) + Generator (Adversarial + λL_{TV})
Latent space	Smooth, Gaussian-regularised (explicit μ, σ)	Implicit; no explicit distribution
Reconstruction quality	Slightly blurry, consistent	Sharper edges (adversarial pressure)
ROI smoothness	Smoother, connected blobs (better)	Sharper but noisier masks
Noise ratio	Lower; prior prevents fragmentation	Higher; adversarial training can fragment
Training stability	Stable, monotonic loss decrease	Sensitive to LR_G / LR_D balance
Interpretability	Latent space inspectable via t-SNE	Black-box; no direct latent access

Key Takeaway: VAE ROIs are smoother and better regularised due to the KL prior. GAN ROIs detect sharper boundaries but are noisier. Ensembling the two difference maps (averaging `D_VAE` and `D_GAN`) typically outperforms either model alone by combining their complementary strengths.

7. Saved Artefacts

7.1 Plot Files

File	Description
training_loss_curves.png	4-panel loss curves (VAE ELBO + GAN G/D)
latent_space_visualization.png	t-SNE scatter + latent histogram
reconstruction_error_plot.png	MSE histograms: normal vs. abnormal
roi_vae.png	VAE 5-column ROI panel (4 samples)
roi_gan.png	GAN 5-column ROI panel (4 samples)
roi_quality_comparison.png	Bar chart: smoothness and noise ratio
edge_pixel_comparison.png	Bar chart: edge pixel counts

7.2 NumPy Tensors and Model Weights

File	Content
npv/vae_roi.npy	VAE binary masks (N, 1, 128, 128)
npv/vae_diff.npy	VAE raw difference maps
npv/vae_abn_orig.npy	Abnormal originals for VAE
npv/vae_abn_recon.npy	VAE reconstructions
npv/gan_roi.npy	GAN binary masks (N, 1, 128, 128)
npv/gan_diff.npy	GAN raw difference maps
npv/gan_abn_orig.npy	Abnormal originals for GAN
npv/gan_abn_recon.npy	GAN reconstructions
vae_weights.pth	Saved VAE model weights
gan_G_weights.pth	Saved GAN generator weights
gan_D_weights.pth	Saved GAN discriminator weights

8. Conclusion

This lab successfully demonstrated unsupervised anomaly detection in brain MRI scans using two generative architectures trained exclusively on normal data. The core assumption was validated: models trained on normal anatomy struggle to reconstruct pathological regions, producing large reconstruction errors localised at tumour sites.

Key Findings:

- 1 **Both VAE and GAN** confirm the anomaly-detection hypothesis: abnormal slices consistently produce higher reconstruction errors than normal slices (Figure 5).

- 2 **VAE latent space** exhibits clear cluster separation between normal and abnormal samples (Figure 4), providing interpretable evidence without anomaly labels.
- 3 **VAE ROIs** (Figure 6) are smoother and less noisy than GAN ROIs (Figure 7), as quantified by lower Laplacian std and noise ratio (Figure 8).
- 4 **GAN ROIs** capture sharper boundaries due to the adversarial loss, at the cost of higher fragmentation.
- 5 **Ensembling** the two difference maps is recommended for best results.

Future Directions:

- β -VAE ($\beta > 1$) for more disentangled latent representations.
- Conditional GAN variants guided by coarse class labels.
- Morphological post-processing of ROI masks (erosion/dilation) to reduce noise while preserving sharp boundaries.

References

-
- [1] D. P. Kingma and M. Welling, 'Auto-Encoding Variational Bayes,' ICLR 2014. arXiv:1312.6114
 - [2] I. Goodfellow et al., 'Generative Adversarial Nets,' NeurIPS 2014. arXiv:1406.2661
 - [3] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, 'Image-to-Image Translation with Conditional Adversarial Networks,' CVPR 2017. arXiv:1611.07004
 - [4] C. Baur et al., 'Deep Autoencoding Models for Unsupervised Anomaly Segmentation in Brain MR Images,' MICCAI Workshop 2018.
 - [5] M. Buda, A. Saha, and M. A. Mazurowski, 'Association of genomic subtypes of lower-grade gliomas with shape features...', Computers in Biology and Medicine, 2019.